Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment No 4

Name: Soham Kulkarni
PRN: 124B1F056
Subject: Design and Analysis of Algorithms
Batch: A3
GitHub Link: https://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

1. Topic

Model a city road network as a weighted graph and route an emergency vehicle from its source to a selected hospital or destination using Dijkstra's algorithm. Use nonnegative travel costs and recompute the route after a road-weight update.

2. Course Objective

- Understand the computational complexity of algorithms.
- Select a suitable algorithm design strategy for a practical problem.

3. Course Outcome

- Find a minimum-cost path using greedy selection and edge relaxation.
- Recover the route, handle weight changes, and compare matrix and heap implementations.

Theory

Dijkstra's algorithm keeps a tentative distance for every vertex. The source starts at zero and other distances start at infinity. Repeatedly select the unvisited vertex with the smallest distance, finalize it, and relax its outgoing edges. Nonnegative edge weights make this greedy finalization safe.

The supplied C++ program uses an adjacency matrix, a linear scan to select the nearest vertex, and a parent array to print the path. Entered roads are undirected. A weight update changes both directions and reruns the algorithm. All weights, including updates, must remain nonnegative.

Working of the Algorithm

- 1. Read nodes, connections, nonnegative weights, source and destination.
- 2. Select the nearest unvisited vertex and try improving each neighbor's distance.
- 3. Record parent vertices when a shorter route is found.
- 4. Print the minimum cost and path, then optionally update a connection and recompute.

Complexity Analysis

Selecting the nearest vertex and scanning its matrix row take O(V) per iteration. Worst-case query time and matrix initialization are O(V squared).

Time Complexity

Best Case: Queries may stop early; matrix setup still takes O(V squared).

Average Case: O(V squared).

Worst Case: O(V squared) per full recomputation.

Space Complexity

O(V squared) for the adjacency matrix and O(V) for distance, parent and visited arrays. The supplied program has a fixed maximum of 1000 nodes.

Algorithm - Steps

- Step 1: Initialize distances, parents and visited flags.
- Step 2: Choose the smallest tentative distance and relax adjacent roads.
- Step 3: Follow parent links to print the route; recompute after a nonnegative update.

Pseudocode

```
set dist[source] = 0; other distances = infinity
repeat up to V-1 times:
    u = unvisited vertex with minimum dist
    if no reachable u exists: stop
    mark u visited
    if u is the destination: stop
    for each unvisited neighbor v:
        if dist[u] + weight(u,v) < dist[v]:
            update dist[v] and parent[v]
print destination cost and the path through parents
```

Flow of Execution

START -> Read graph -> Select nearest vertex -> Relax edges -> Print path -> Update and rerun if needed -> END.

6. Example

For the saved five-node network, source is 0 and destination is 4. The optimal path is 0 -> 2 -> 1 -> 3 -> 4, with cost 2 + 1 + 5 + 2 = 10. Node IDs start at 0 in this program.

Adjacency lists and a binary heap can improve sparse-network time to O((V+E) log V). BFS handles unweighted graphs; Bellman-Ford supports negative edges. This program uses a selected destination.

Conclusion

Dijkstra's algorithm finds a shortest route for nonnegative weights. Parent links recover the path, and recomputation handles updated costs. A heap-based version improves scalability.